Solution: Closest Binary Search Tree Value (medium)

Problem Statement

Given a binary search tree (BST) and a target number, find a node value in the BST that is closest to the given target.

A BST is a tree where for every node, the values in the left subtree are smaller than the node, and the values in the right subtree are greater.

Examples:

Example 1:

Input:

```
   Tree:
      5
    /   \
   3     8
  / \   / \
 1   4 6   9
```

Target: 6.4

Expected Output: 6

Justification: The values 6 and 8 are the closest numbers to 6.4 in the tree. Among them, 6 is closer.

Example 2:

Input:

```
   Tree:
      20
    /    \
   10     30
```

Target: 25

Expected Output: 20

Justification: 20 and 30 are the closest numbers to 25. However, 20 is closer than 30.

Example 3:

Input:

```
   Tree:
      2
    /   \
   1     3
```

Target: 2.9

Expected Output: 3

Justification: 3 is the closest value to 2.9 in the tree.

Solution

To solve the problem, initialize the closest value with the root node's value. Then, iteratively traverse the tree, comparing the target with the current node's value. If the target is closer to the current node than the closest value found so far, update the closest value. Depending on whether the target is greater or smaller than the current node's value, move to the right or left child, respectively. This process continues until all nodes are traversed.

To solve this problem, we can leverage the properties of a BST. As we traverse the BST, we'll always choose the path that brings us closer to the target value. As we're doing so, we'll keep track of the closest node value we've encountered so far.

1. Initialize the closest value (`closestVal`) with the root's value.
2. Traverse the BST starting from the root. At each node:
   - Update the `closestVal` if the current node's value is closer to the target than the previously stored `closestVal`.
   - If the target value is greater than the current node's value, move to the right child. Otherwise, move to the left child.
3. Return `closestVal` after traversing the tree.

The reason this approach is effective is that it utilizes the BST's property to eliminate half of the tree's possibilities at each step, ensuring we follow the path that may contain the closest value.

Algorithm Walkthrough:

Using Example 1:

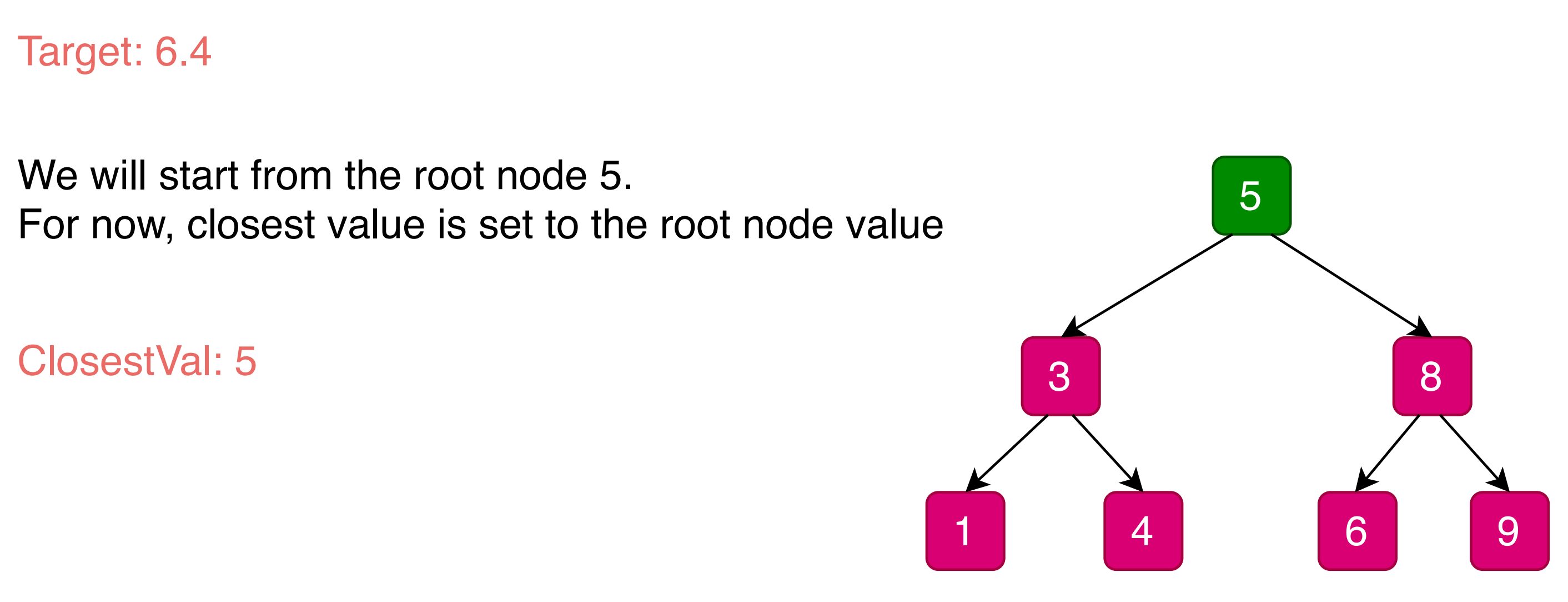
Input:

```
   Tree:
      5
    /   \
   3     8
  / \   / \
 1   4 6   9
```

Target: 6.4

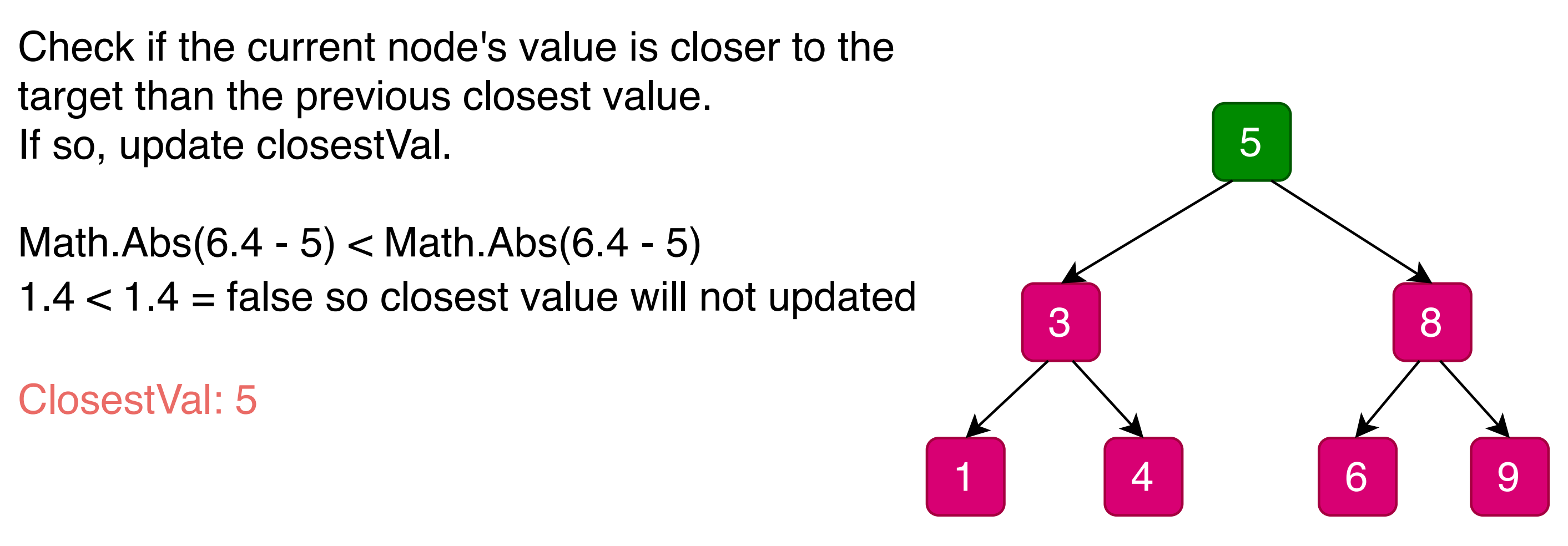
- Start at the root (5). Current `closestVal` = 5.
- Target (6.4) > Current node (5), move right.
- Arrive at node 8. Since 6.4 is closer to 6 than 5, update `closestVal` to 6.
- Target (6.4) < Current node (8), move left.
- Arrive at node 6. No more children to check.
- Return `closestVal`, which is 6.

Here is the visual representation of the algorithm:


Target: 6.4

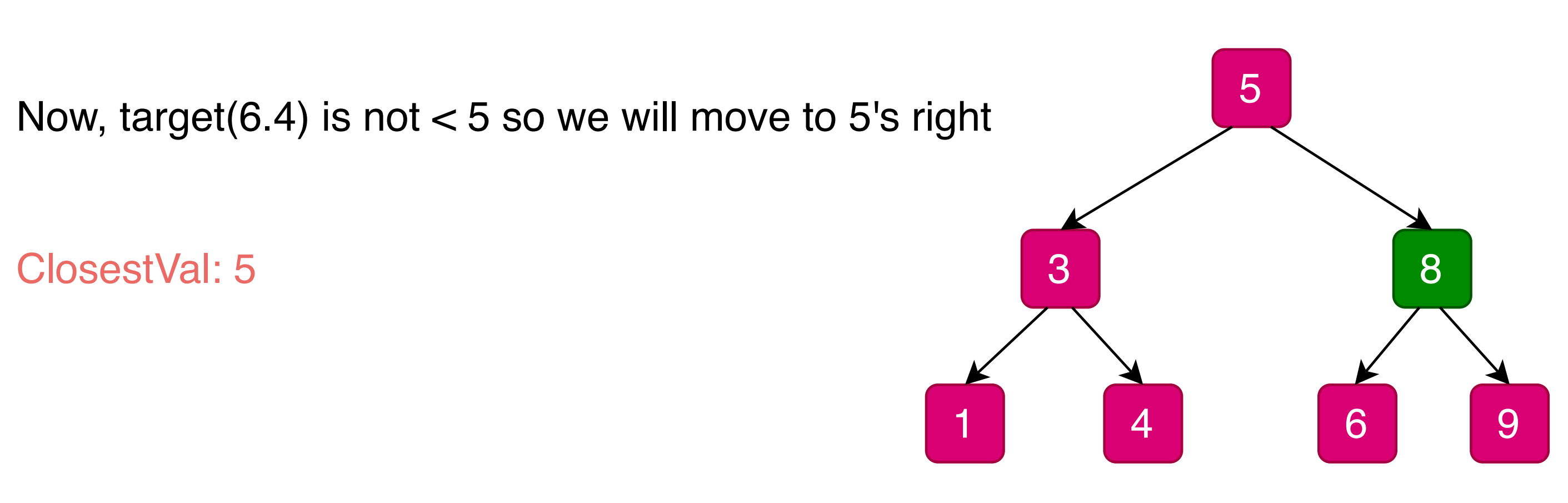
We will start from the root node 5.
For now, closest value is set to the root node value

ClosestVal: 5

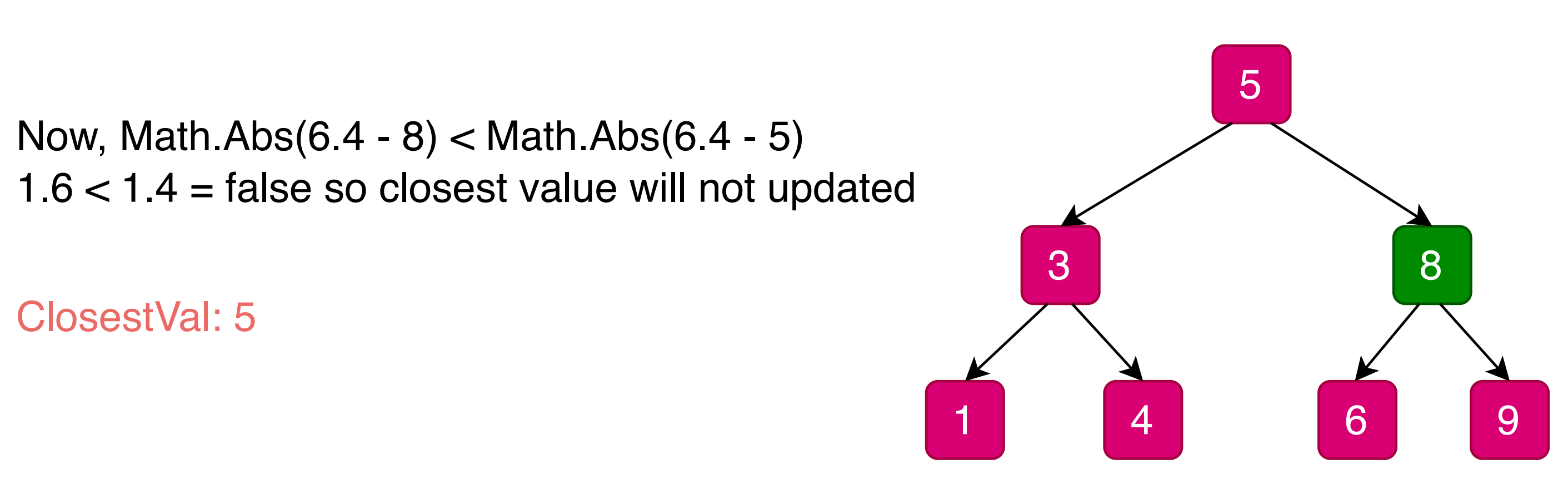

Check if the current node's value is closer to the target than the previous closest value.
If so, update closestVal.

Math.Abs(6.4 - 5) < Math.Abs(6.4 - 5)
1.4 < 1.4 = false so closest value will not updated

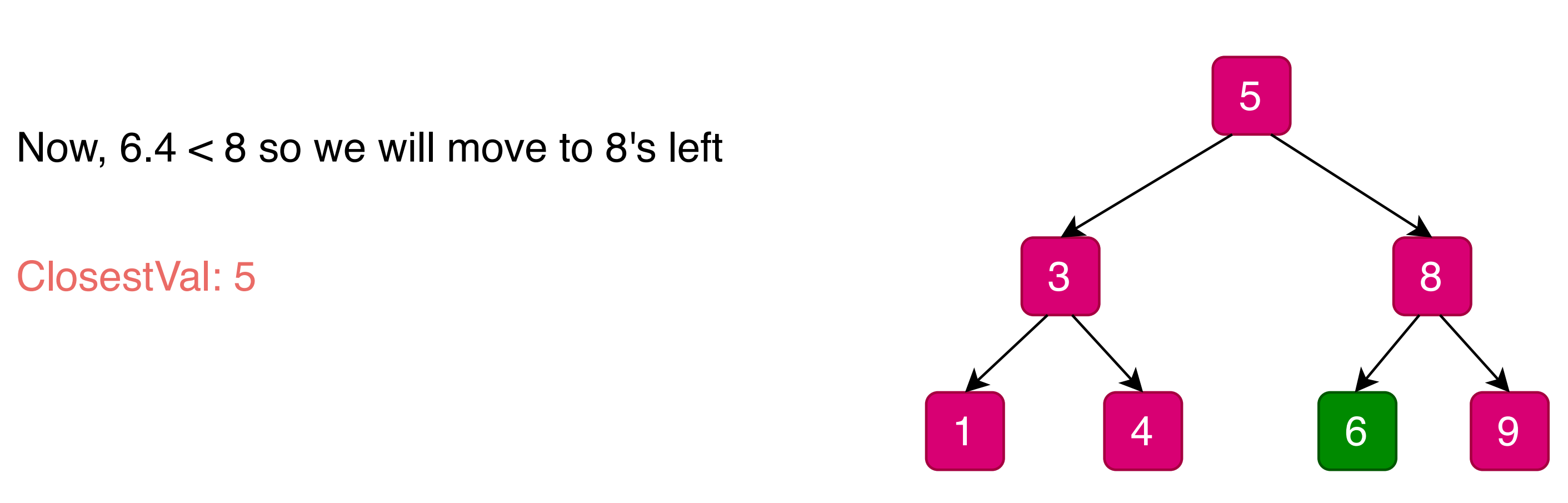
ClosestVal: 5


Now, target(6.4) is not < 5 so we will move to 5's right

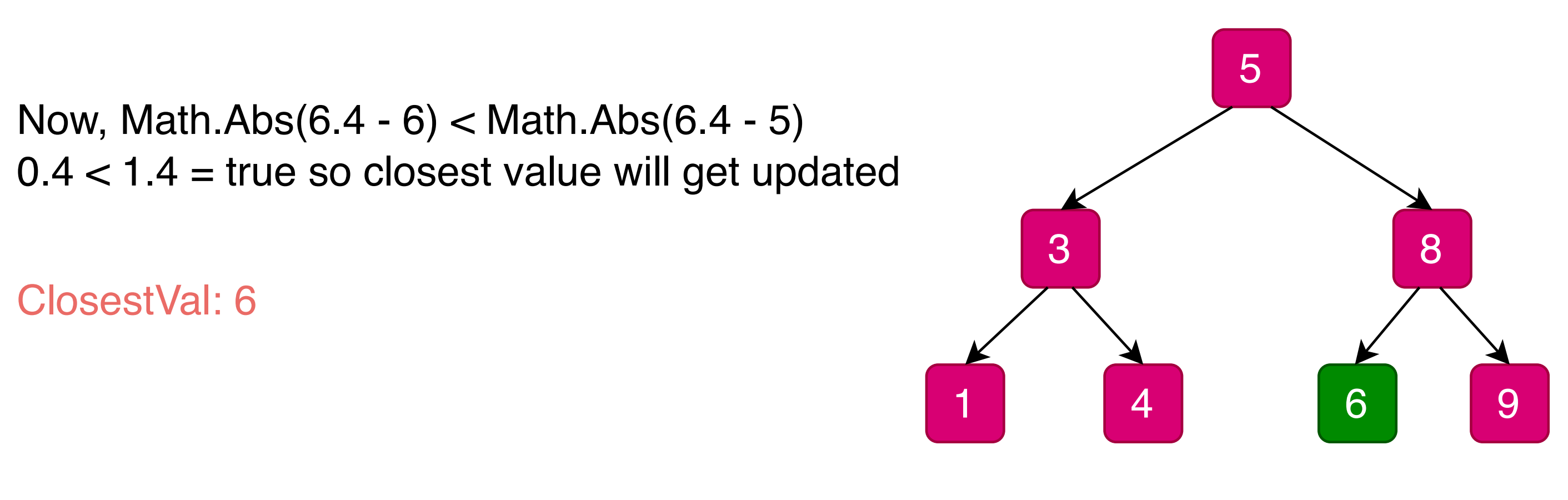
ClosestVal: 5


Now, Math.Abs(6.4 - 8) < Math.Abs(6.4 - 5)
1.6 < 1.4 = false so closest value will not updated

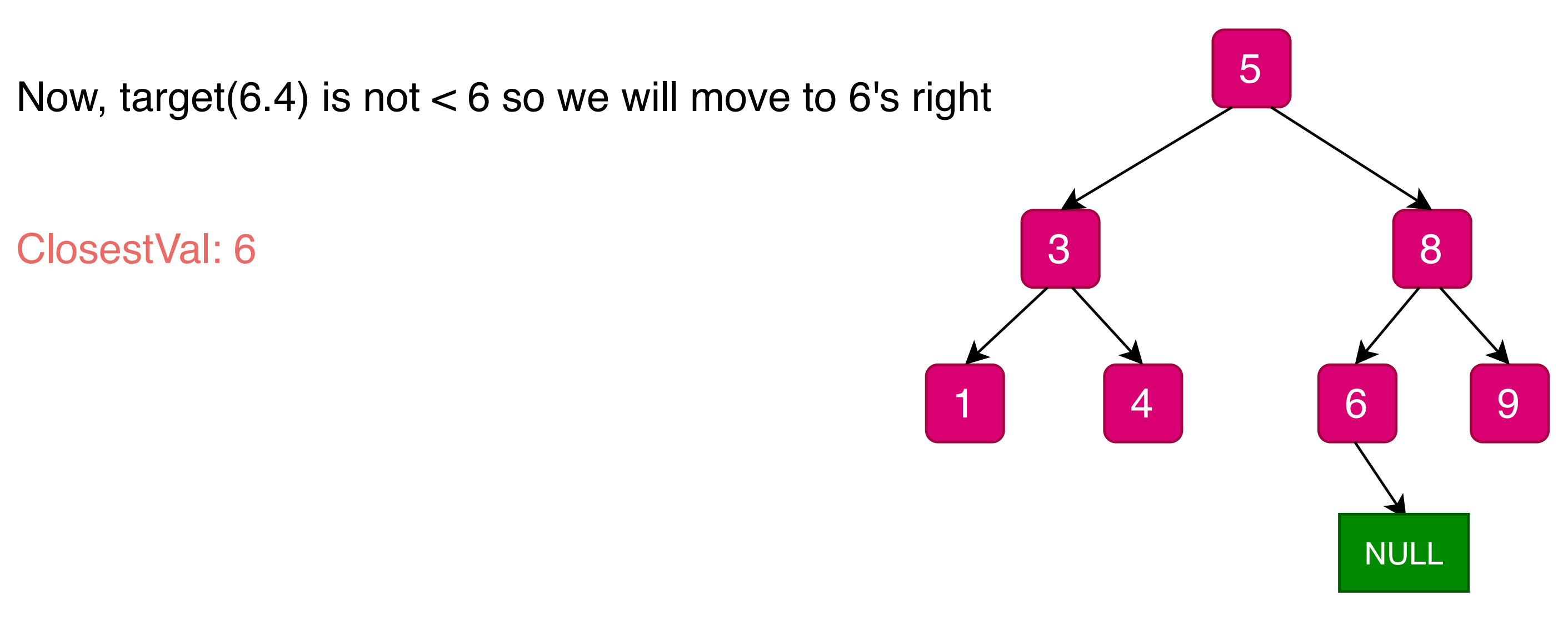
ClosestVal: 5


Now, 6.4 < 8 so we will move to 8's left

ClosestVal: 5


Now, Math.Abs(6.4 - 6) < Math.Abs(6.4 - 5)
0.4 < 1.4 = true so closest value will get updated

ClosestVal: 6


Now, target(6.4) is not < 6 so we will move to 6's right

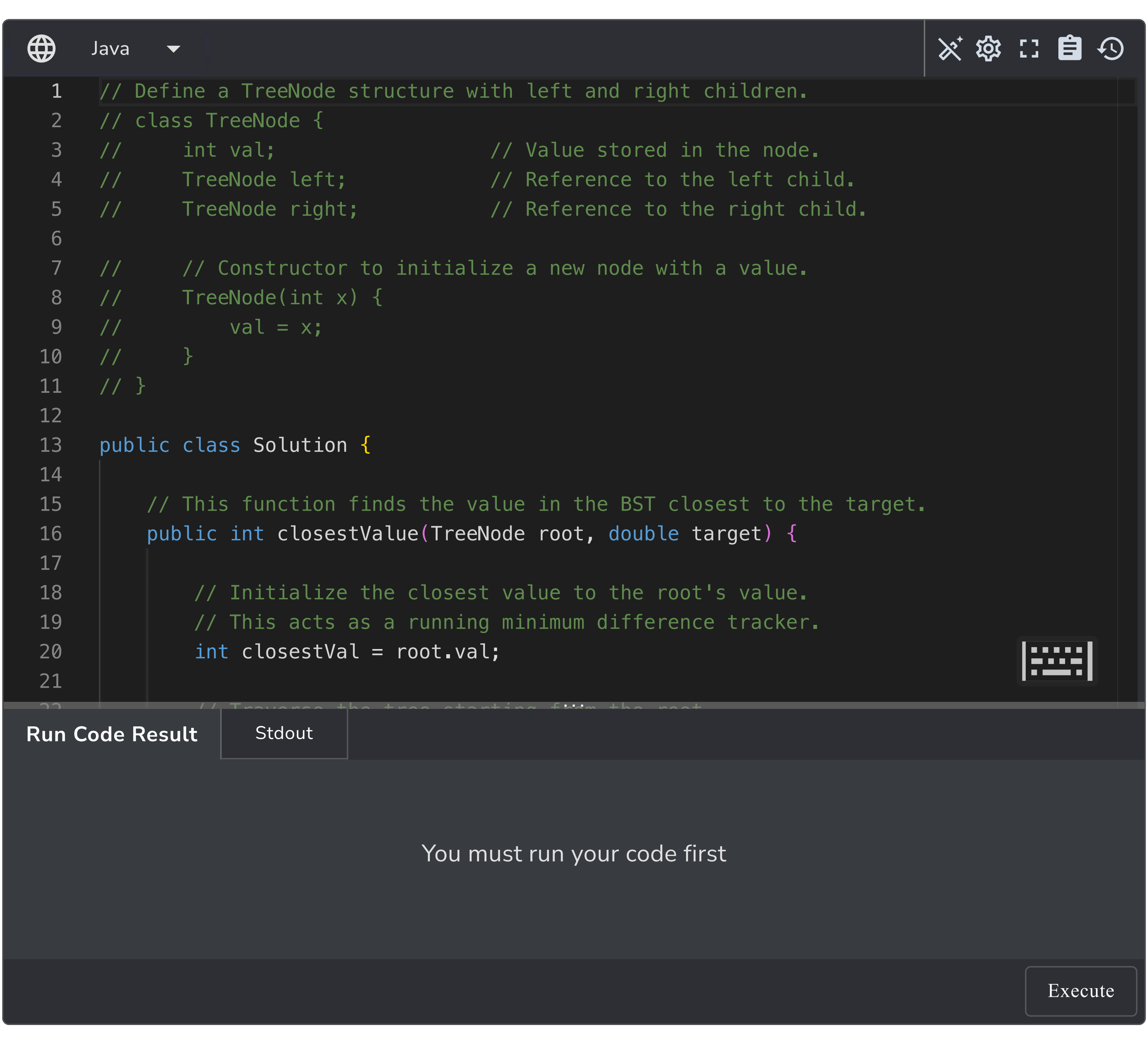
ClosestVal: 6

Output: 6 will be returned as we have no further nodes

Closest BST Value

Code

Here is the code for this algorithm:



```java
// Define a TreeNode structure with left and right children.
// class TreeNode {
//     int val;                    // Value stored in the node.
//     TreeNode left;              // Reference to the left child.
//     TreeNode right;             // Reference to the right child.

//     // Constructor to initialize a new node with a value.
//     TreeNode(int x) {
//         val = x;
//     }
// }

public class Solution {

    // This function finds the value in the BST closest to the target.
    public int closestValue(TreeNode root, double target) {

        // Initialize the closest value to the root's value.
        // This acts as a running minimum difference tracker.
        int closestVal = root.val;

```

You must run your code first

Complexity Analysis

Time complexity: $O(logn)$ on average, because in a balanced BST, we'll typically only need to traverse down the height of the tree. In the worst-case scenario of a skewed tree, it will be $O(n)$. Space complexity: $O(1)$ because we only use a constant amount of space to store the `closestVal`.

← Previous
Closest Binary Search Tree Value (Medium)

Next →
Merge Two Binary Trees (Medium)

☐ Mark as Completed